Sprint 10 · Reto «¿Vale la pena la red?»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. La dirección de TurisData oye hablar de **redes neuronales** en todas partes y pregunta si debe **sustituir su modelo de cancelaciones por una red**. No quiere una opinión: quiere **evidencias, con el coste incluido**, para decidir si merece la pena el cambio. La pregunta del reto es *¿una red neuronal mejora al aprendizaje automático clásico, y a qué precio?*, y la respuesta puede ser **«no»**: un «no compensa» bien demostrado es tan válido y útil como un «sí». Vais a entrenar una red sencilla (un MLP, una red de capas de neuronas artificiales), a compararla con un modelo clásico de árboles (el *boosting*) y a escribir un **informe de 1–2 páginas** que termine en «Recomendamos… porque…».

Cómo se trabaja. No hace falta experiencia previa: os damos un **cuaderno de Colab casi hecho** (`S10_04` , con cuatro ejercicios con hueco y comprobaciones automáticas que os dicen si lo que escribís es correcto) y vuestro trabajo es completarlo, probar variantes, mirar las curvas, comparar con honestidad (con varias semillas, para que un número más alto no os engañe) y contarlo en el informe. En clase tenéis 10 minutos de planning (día 1) y unos **90 minutos de reto + 40 de entrega, review y retro** (día 2). Los seis pasos suman 145 minutos de trabajo: lo que no quepa en clase va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
El cuaderno del reto `S10_04_reto_vale_la_pena`	Es el esqueleto: partición, boosting de referencia, MLP con 3 semillas, curvas, comparación con intervalo y la variante de precio	Colab (basta con CPU; no hace falta GPU). Su código explicado lo cuenta celda a celda
Cuadernos de práctica `S10_02` y `S10_03`	Las pistas: cómo se construye un MLP en Keras y cómo se ven y se combaten el sobreajuste, el *dropout*, la L2 y la parada temprana	Colab; su código explicado
Presentación `S10.3` y apuntes de redes neuronales	Repasar las curvas de aprendizaje y la regularización (las diapositivas 6–7 son de este reto)	Carpeta de teoría del sprint
Los datos `reservas_turisdata.csv`	6000 reservas **sintéticas** (inventadas), con la columna `cancelada` y el `precio_noche`	El cuaderno los descarga solo en su primera celda; no hace falta tocarlos
Google Colab (Python, Keras y scikit-learn ya instalados)	Ejecutar el cuaderno sin instalar nada	colab.research.google.com; al terminar, *Archivo → Descargar → Descargar .ipynb*

Recurso	Para qué lo usaréis	Dónde está
Cómo entregar en GitHub	Subir el cuaderno, el informe y el README a `sprint-10/` desde el navegador	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender conceptos, curvas y errores de Keras	Con las reglas de la «Ayuda 1» del paso a paso

3 · Guía paso a paso

Son seis pasos, de **145 minutos en total**. Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. En el horario del día 2 caben unos 105 minutos (90 de reto y 15 para poneros al día); los 40 que faltan se hacen en la **consolidación** (13:00–13:30) y en la **6.ª hora** (hasta las 14:30), y la subida puede hacerse hasta esa hora. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa. Trabajad siempre en **una copia** del cuaderno que sea vuestra, sin tocar el original del curso.

Paso 1 · Elegir el problema y reproducir la referencia (día 2, 15 minutos)

1. Elegir **un problema**: cancelaciones (`cancelada`) o precio (`precio_noche`).
2. Abrir `S10_04` en Colab y ejecutarlo bloque a bloque, entendiendo cada uno.
3. Reproducir el modelo de referencia y anotar su métrica y su tiempo.
4. Fijar la partición entrenamiento / validación / prueba y no cambiarla nunca más.

Cómo se hace

- **Elegir el problema.** El cuaderno trae montado el de cancelaciones, así que es el camino más corto. Si elegís precio, seguid la sección 7 del cuaderno (hay que **quitar `precio_noche` de las entradas**). El otro problema es opcional.
- **Abrir el cuaderno.** Pulsad el enlace de Colab y haced una copia (*Archivo → Guardar una copia en Drive*). **Ejecutad siempre primero la celda «Preparación»**, que descarga los datos.
- **Ir bloque a bloque.** Pulsad ▶ en cada celda y leed lo que pone antes. Los ejercicios 1 a 4 tienen un hueco (`...`) y dan error hasta que lo rellenéis: es lo normal. En este paso, haced el **ejercicio 1** (`evaluar`), la función que mide a todos los modelos por igual. Si no entendéis un bloque, preguntad antes de seguir.
- **La partición.** La sección 1 reparte 64 % entrenamiento, 16 % validación y 20 % prueba, con semilla fija. No toquéis ni la semilla ni los `random_state`. La validación sirve para **decidir**; la prueba, para el veredicto final y **una sola vez**.
- **La referencia.** La sección 3 entrena el boosting (`HistGradientBoostingClassifier`). Si vuestro equipo tiene un campeón del Sprint 8 (XGBoost, LightGBM, Random Forest…), usadlo en su lugar y compararéis con él. **No se cambia la referencia para que la red gane.**

Qué deberíais ver. Al ejecutar la partición, tres tamaños: `(3840, 35) (960, 35) (1200, 35)`. Al entrenar el boosting, una línea como `Boosting entrenado en 0.3 s · AUC validación = 0.73`. Un AUC de 0,5 sería lo mismo que tirar una moneda; uno cercano a 1,0 haría sospechar de una fuga de datos.

Hecho cuando: el cuaderno llega hasta el boosting sin errores, el ejercicio 1 pasa su comprobación y tenéis anotadas la métrica y el tiempo de la referencia.

Paso 2 · Primera red (día 2, 25 minutos)

1. Comprobar que los datos están escalados **ajustando el escalador solo con el entrenamiento**.
2. Entrenar un MLP pequeño en CPU, con un máximo de 40 épocas.
3. Dibujar la curva de aprendizaje: error en entrenamiento y en validación.
4. Hacer el ejercicio 2 (el AUC de cada semilla).

Cómo se hace

- **Escalar sin fugas.** Escalar es poner todas las columnas en una escala parecida para que la red aprenda bien. El cuaderno ya lo hace en la sección 1 con `pre.fit_transform(X_ent)` (solo entrenamiento) y `pre.transform(...)` para validación y prueba. Mirad por qué: si el escalador viera la prueba, la red «se enteraría» de ella (eso es una **fuga**).
- **El MLP.** La sección 4 define `construir_mlp`: dos capas ocultas (64 y 32 neuronas), L2, *dropout*, parada temprana y un máximo de 40 épocas. Si preferís algo todavía más pequeño (por ejemplo, 2 capas de 32 neuronas y unas 30 épocas), cambiad los argumentos `capas` y `epochs`. Se entrena con **3 semillas**, y en CPU tarda pocos minutos.
- **Las curvas.** La sección 5 dibuja, para cada semilla, el error en entrenamiento y en validación a lo largo de las épocas. Mirad también cuántas épocas hizo cada una antes de parar.
- **Ejercicio 2.** Calculad en `aucs_mlp_val` el AUC de cada modelo sobre la **validación** y en `auc_mlp_medio` su media.

Qué deberíais ver. Tres líneas, una por semilla, del estilo `semilla 1: 17 épocas, AUC validación = 0.7… (máx.)`, tres gráficas con dos curvas cada una y, tras el ejercicio 2, `AUC de validación por semilla: …` con la media. La comprobación del cuaderno espera una media entre 0,6 y 0,85.

Hecho cuando: hay un MLP entrenado con su curva de aprendizaje, el ejercicio 2 pasa su comprobación y sabéis decir en una frase qué hace una época.

Paso 3 · Variantes (día 2, 25 minutos)

1. Probar **2 o 3 variantes** del MLP cambiando un solo ajuste cada vez.
2. Compararlas mirando **solo la validación**: el test no se toca.
3. Anotar en una tabla pequeña qué ha mejorado y qué no.

Cómo se hace

- **Qué cambiar.** `construir_mlp` ya tiene los botones: `capas` (más o menos neuronas, por ejemplo `(32,)` o `(128, 64)`), `dropout` (por ejemplo `0.5`), `l2` (por ejemplo `0.01`) y `tasa` (la tasa de aprendizaje, por ejemplo `3e-4`). También podéis probar otra paciencia en la parada temprana. **Cambiad una sola cosa por variante**, para saber qué ha causado el efecto.
- **Cómo comparar.** Entrenad cada variante y fijaos en la pérdida (el error) y el AUC **de validación**. Una tabla basta:

Variante	Qué cambia	Error en entrenamiento	Error en validación	AUC en validación
Base	`capas=(64, 32)`, `dropout=0.3`, `l2=0.003`	(vuestra cifra)	(vuestra cifra)	(vuestra cifra)
Variante 1	(lo que cambiáis)	(vuestra cifra)	(vuestra cifra)	(vuestra cifra)
Variante 2	(lo que cambiáis)	(vuestra cifra)	(vuestra cifra)	(vuestra cifra)

- **Cómo elegir.** Quedaos con la variante que mejor salga **en validación**. Si dos quedan casi iguales, escoged la más sencilla. Si miráis la prueba para decidir, ya no es una prueba.

Qué deberíais ver. Una tabla con la base y las 2–3 variantes, y una frase por variante: «mejoró», «empeoró» o «igual», con el número al lado. Que una variante no mejore es un resultado válido: lo importante es apuntarlo.

Hecho cuando: tenéis la tabla rellena, habéis elegido una variante mirando solo la validación y sabéis explicar por qué.

Paso 4 · Diagnosticar el sobreajuste (día 2, 20 minutos)

1. Mirar las curvas de vuestra mejor red.
2. Hacer el ejercicio 3 (`diagnostico`).
3. Escribir una frase de diagnóstico.

Cómo se hace

- **Leer las curvas.** Hay tres formas típicas:
- **Entrenamiento baja y validación sube:** la red memoriza (sobreajuste). Se arregla con más regularización, menos neuronas o parando antes.
- **Las dos curvas están altas y juntas:** la red es demasiado simple (subajuste).

- **Las dos bajan y se estabilizan juntas:** buen ajuste.
- **Ejercicio 3.** Escribid `diagnostico(historial, margen=0.03)`: devuelve `"sobreajuste"` si la pérdida de validación **final** supera a su **mínimo** en más de `margen`, o si la de entrenamiento final es menor que la de validación final en más de `0.15`; si no, `"sin sobreajuste claro"`. Los `assert` del cuaderno os dicen si va bien. Aplicadlo a las tres semillas.
- **La frase.** Una frase para vuestra mejor red, con una cifra o una época: «Con la variante X, la validación deja de mejorar en la época 11 y el entrenamiento sigue bajando: hay un poco de sobreajuste, que la parada temprana corta a tiempo».

Qué deberíais ver. Tres líneas `MLP semilla 1: …` con `sobreajuste` o `sin sobreajuste claro`, y vuestra frase escrita junto a la gráfica.

Hecho cuando: el ejercicio 3 pasa su comprobación y la frase de diagnóstico está escrita (en el cuaderno y, resumida, en el informe).

Paso 5 · Comparación justa (día 2, 25 minutos)

1. Repetir el entrenamiento con varias semillas y calcular media ± desviación.
2. Medir el veredicto en la prueba **una sola vez**, con el MLP elegido y con la referencia.
3. Hacer el ejercicio 4 (¿la diferencia es real o es ruido?).
4. Añadir el coste: tiempo, complejidad y explicabilidad.

Cómo se hace

- **Varias semillas.** El bucle de la sección 4 ya entrena con tres semillas (1, 2 y 3). Si el MLP elegido es una variante, **cambiad en ese bucle los argumentos de `construir_mlp`** y volved a ejecutar antes de seguir. Calculad la media y la desviación del AUC; si el modelo clásico da siempre exactamente la misma cifra, el intervalo *bootstrap* del cuaderno es vuestra medida de incertidumbre.
- **El veredicto.** La sección 6 usa la prueba **una sola vez**: media de las 3 semillas para el MLP y el boosting, con la función `evaluar` del ejercicio 1. Apuntad los resultados y no la volváis a mirar para cambiar nada.
- **¿Real o ruido?** El *bootstrap* remuestrea la prueba 300 veces y da un intervalo para la diferencia de AUC. **Si el intervalo incluye el 0, no podéis decir que uno sea mejor.** Eso es el **ejercicio 4**: `hay_diferencia` (verdadero si el intervalo no contiene el 0) y `mejor_modelo` (`"MLP"`, `"Boosting"` o `"Empate"`).
- **El coste.** La tabla del cuaderno ya trae el tiempo de entrenamiento y el número de parámetros. Añadid la **explicabilidad** (explicar por qué un boosting dice «cancelará» es más fácil que explicar una red). Una red puede mejorar un 1 % y tardar 20 veces más.

Qué deberíais ver. Una tabla con dos filas (`Boosting de árboles` y `MLP (media de 3 semillas)`) y las columnas AUC, AP, F1, precisión, tiempo y parámetros; una línea `Diferencia de AUC (MLP − boosting): … · intervalo 95 %: […]`, un histograma con una línea roja en el 0 y `Veredicto estadístico: …`. Con estos datos lo esperable es que los dos queden muy cerca, y un «Empate» es un resultado perfectamente válido.

Hecho cuando: el ejercicio 4 pasa su comprobación, tenéis media ± desviación (o intervalo) de ambos enfoques y los números del coste apuntados.

Paso 6 · Informe, revisión y entrega (día 2, 35 minutos)

1. Escribir el informe de 1 página, con la recomendación al principio.
2. Rellenar el README de `sprint-10/`.
3. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado al final del informe.
4. Subir los tres ficheros del equipo a `sprint-10/`, y cada persona su autoevaluación.

Cómo se hace

- **El informe** (`informe_comparativo.md`). Seis partes, en este orden:

Parte	Qué escribís	De dónde sale
1. Pregunta y respuesta	Una frase: «Recomendamos mantener / cambiar a… porque…»	Vuestra decisión del paso 5
2. Tabla de resultados	Media ± desviación de ambos enfoques (AUC, AP, F1)	Secciones 4 y 6 del cuaderno
3. Curva comentada	Qué se ve y vuestra frase de diagnóstico	Paso 4
4. Coste	Tiempo, complejidad y explicabilidad	Paso 5
5. Riesgos de IA responsable	Al menos dos, específicos, con su medida (datos sintéticos; umbral y coste de los errores por *overbooking*; sesgos por país o canal)	Vuestra reflexión
6. Límites	Qué no sabemos y qué haríamos después	Vuestra reflexión

Si queréis ver la curva en el informe, guardad la imagen del cuaderno como `curvas_aprendizaje.png` y subidla a `sprint-10/`. Añadid también una línea de **uso de la IA**: qué pedisteis y qué comprobasteis a mano.

- **El README.** Abrid `sprint-10/README.md` → lápiz y completadlo: *Reto* («¿Vale la pena la red?»), *Qué se entrega* (el cuaderno y el informe), *Quién hizo qué* y *Cómo reproducirlo* («abrir en Colab y ejecutar de arriba abajo»).
- **La revisión con la rúbrica (5 minutos).** Quien revisa sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica y añade al final del informe esta sección, con un nivel de 1 a 4 por criterio y **una frase de prueba**:

```
## Revisión con la rúbrica

Revisado por: (nombre) · Fecha: (dd/mm/aaaa)

| Criterio | Nivel (1–4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | | |
| Reproducibilidad | | |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

Lo primero que mejoraríamos: (una cosa concreta)
```

Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos». Si sobran minutos, empezad un extra (la variante de precio, o guardar el modelo y recargarlo).

- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub. Para el cuaderno: en Colab, *Entorno de ejecución → Reiniciar y ejecutar todo* y después *Archivo → Descargar → Descargar .ipynb* (renombradlo a `reto_vale_la_pena.ipynb`). En el repositorio del equipo: (1) **entrad primero en la carpeta `sprint-10`**; (2) *Add file → Upload files*; (3) arrastrad el fichero; (4) escribid un mensaje corto y pulsad **Commit changes**. El informe también se puede crear directamente con *Add file → Create new file*. Cada subida es un *commit* con el nombre de quien la hace, así que repartid: una persona sube el cuaderno, otra escribe el informe y otra rellena el README. El CSV va a `data/` solo si aún no está.

Qué deberíais ver. En la carpeta `sprint-10` del repositorio, el cuaderno con sus resultados visibles, el informe, el README relleno y una hoja de autoevaluación por persona (**3 + N**); y en el historial (el icono del reloj), *commits* de personas distintas.

Hecho cuando: otra persona podría repetir el cuaderno siguiendo solo el README, el informe termina con «Recomendamos… porque…» y trae la sección «Revisión con la rúbrica» rellena, y los ficheros están en `sprint-10/`.

Después, en la entrega y la review: cada persona rellena y sube su autoevaluación y coevaluación a `sprint-10/`, y el equipo enseña, en unos 5 minutos, la tabla comparativa, una curva de aprendizaje comentada y la recomendación en una frase, con el coste y los riesgos.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **explica con mucha seguridad cosas que pueden estar mal**. Reglas:

- **Solo se compara con las mismas métricas y la misma partición.** No vale cambiar la referencia para que la red gane.
- **Las variantes se eligen con la validación**, nunca con el test.
- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una celda, tenéis que saber explicarla.
- **Declaradlo** en el informe: dónde habéis usado IA y qué habéis comprobado. **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender conceptos	«Explícame qué es el sobreajuste en una red neuronal con una analogía de estudiar para un examen memorizando»	Que lo explicáis con vuestras palabras
Interpretar una curva	«Mi pérdida de entrenamiento baja pero la de validación sube a partir de la época 12. ¿Qué significa y qué puedo probar?»	Que probáis lo que sugiere y comprobáis el efecto
Entender un error de Keras	«Me da este error: *[mensaje]* con este código: *[código]*»	Que entendéis la causa
Revisar el informe	«Léelo como un directivo y dime qué no se entiende»	Que no cambie cifras

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Escalar con todos los datos	Se olvida la fuga	`fit` del escalador solo con el entrenamiento
Mirar el test para elegir la variante	«Para saber cuál va mejor»	Solo validación; el test, una sola vez
Comparar con una sola semilla	«Ya ha salido mejor»	Varias semillas y media ± desviación
Entrenar 500 épocas	«Cuanto más, mejor»	Máximo 40 épocas y parada temprana
Concluir «la red es mejor» por 0,3 puntos	Se mira solo el número	Comparad los intervalos y añadid el coste
Informe sin recomendación	Se queda en los números	Terminad con «Recomendamos… porque…»
Una celda da error con `...`	Falta rellenar el hueco de un ejercicio	Completad los ejercicios en orden, del 1 al 4
Subir el cuaderno sin resultados	Se descarga sin haberlo ejecutado	*Reiniciar y ejecutar todo* y después descargar el `.ipynb`

Ayuda 3 · Si os atascáis

1. **La red no aprende (la pérdida no baja):** revisad que los datos estén escalados y probad una tasa de aprendizaje menor.
2. **Las curvas son ruidosas:** es normal en datos pequeños; usad varias semillas y promediad.
3. **No entendemos las curvas:** repasad `S10_03_regularizacion`, que tiene un ejemplo comentado, y el código explicado de `S10_04`.
4. Preguntad al equipo y, después, al docente. Más ayuda: Cómo se trabaja un reto.